

stale resource: stale resource is any cloud resource that is no longer actively used or needed but continues to exist in the environment, consuming storage or incurring costs. Examples include unattached EBS volumes, orphaned snapshots, unused Elastic Load Balancers, and expired security groups. These resources can clutter your cloud environment, complicate resource management, and lead to unnecessary expenses. Deleting stale resources is essential to optimize costs, enhance operational efficiency, and reduce environmental impact by minimizing wasted compute and storage resources. Regularly identifying and cleaning up stale resources is a best practice for maintaining a secure, cost-effective, and sustainable cloud infrastructure.

Clean Up EBS Volumes and Snapshots:

- **Unattached Volumes:**
 - Occur when EC2 instances are terminated without deleting associated volumes.
 - Continue accruing charges.
- **Orphaned Snapshots:**
 - Exist when snapshots of deleted EBS volumes remain in the account.
 - Consume S3 storage and incur costs.
- **Best Practices:**
 - Regularly review and clean up unused volumes and snapshots.

Stale resources remain in your cloud environment due to operational oversights, lack of automated cleanup processes, or as remnants of testing and development activities. Here's why they may exist and continue to consume resources:

1. **Unattached EBS Volumes:**

- When an EC2 instance is terminated without deleting attached volumes, the EBS volumes remain in the **available** state. These volumes are still billed based on their size, even though they are not actively in use.

2. **Orphaned Snapshots:**

- Snapshots taken as backups of deleted volumes persist in Amazon S3. Although they are incremental, they continue to consume S3 storage and incur costs.

3. **Unused Elastic IPs and Load Balancers:**

- Elastic IPs not attached to an instance and unused load balancers can remain active and incur charges.

4. **Expired Security Groups or Rules:**

- Security configurations no longer linked to resources can clutter the environment, making management complex and increasing the attack surface.

Why Deleting Stale Resources is Important:

- **Cost Savings:** Prevents unnecessary billing for unused resources.
- **Operational Efficiency:** Reduces clutter, making resource management easier and more effective.
- **Security:** Limits exposure by removing unused or misconfigured security groups and other resources.
- **Environmental Sustainability:** Reduces the cloud provider's energy consumption by releasing unutilized compute and storage resources.

Created stack using cloud formation

EC2 S3 CodeDeploy CodePipeline RDS VPC IAM CloudFormation

CloudFormation > Stacks > stale-stack

CloudFormation <

- Stacks
 - Stack details
 - Drifts
- StackSets
- Exports

Infrastructure Composer

laC generator

Hooks overview [New](#)

Hooks [New](#)

- Registry
- Public extensions
 - Activated extensions
 - Publisher

Stacks (1)

Filter status

☒ View nested

Active 

< 1 >

Stacks

 stale-stack
2024-12-22 22:09:56 UTC+0530
 CREATE_COMPLETE

stale-stack

- [Delete](#) [Update](#) [Stack actions](#)  [Create stack](#) 
- < [Stack info](#) [Events - updated](#) [Resources](#) [Outputs](#) [Parameters](#) [Template](#) >

[Table view](#) [Timeline view - new](#)

Events (15)

Timestamp	Logical ID	Status	Detailed status
2024-12-22 22:10:19 UTC+0530	stale-stack	 CREATE_COMPLETE	-
2024-12-22 22:10:18 UTC+0530	WebAppVolumeAttachm ent	 CREATE_COMPLETE	-
2024-12-22 22:10:14 UTC+0530	stale-stack	 CREATE_IN_PROGRES S	 CONFIGURATION_CO Mplete
2024-12-22 22:10:13 UTC+0530	WebAppVolumeAttachm ent	 CREATE_IN_PROGRES S	 CONFIGURATION_CO Mplete

Instance created with the help of stack

EC2

S3

CodeDeploy

CodePipeline

RDS

VPC

IAM

CloudFormation

Dashboard

EC2 Global View

Events

Instances

Images

Elastic Block Store

Network & Security

Load Balancing

Instances (1/1) Info

Last updated 1 minute ago

Connect

Instance state

Actions

Launch instances

Find Instance by attribute or tag (case-sensitive)

All states

	Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4
☒	webapp	i-071f12f1416e9ad45	Running	t2.micro	2/2 checks passed	View alarms +	us-east-1a	ec2-18-207

i-071f12f1416e9ad45 (webapp)

root device name

/dev/xvda

root device type

EBS

EBS optimization

disabled

Block devices

Filter block devices

	Volume ID	Device name	Volume size (GiB)	Attachment status	Attachment time	Encrypted	KMS key ID
☒	vol-089f398b4da6180cd	/dev/xvda	8	Attached	2024/12/22 22:10 GMT+5:30	No	-
☐	vol-0ae60e8b916b8cf86	/dev/xvdf	4	Attached	2024/12/22 22:10 GMT+5:30	No	-

Volume monitoring (1)

Ebs volumes created

EC2 > Spot requests

Dashboard

EC2 Global View

Events

Instances

Instances

Instance Types

Launch Templates

Spot Requests

Savings Plans

Reserved Instances

Dedicated Hosts

Capacity Reservations

Images

AMIs

AMI Catalog

Elastic Block Store

Volumes

Snapshots

Lifecycle Manager

Volumes (2) Info

Saved filter sets

Choose filter set

Q Search

Actions

Create volume

	Name	Volume ID	Type	Size	IOPS	Throughput	Snapshot ID	Created
☐	webapp-ebs	vol-0aa16eb80f3d0eab7	gp2	8 GiB	100	-	snap-00e5da2...	2024/12/23 01:06 GMT+5:...
☐	new-ebs	vol-03eca3634f9f8d302	gp3	4 GiB	3000	125	-	2024/12/23 01:07 GMT+5:...

Fault tolerance for all volumes in this Region

Snapshot summary

Recently backed up volumes / Total # volumes

0 / 2

Last updated on Mon, Dec 23, 2024, 12:12:25 AM (GMT+05:30)

Data Lifecycle Manager default policy for EBS Snapshots status

No default policy set up | Create policy

Unable to create Snapshots in CloudFormation

Snapshots are not automatically created in AWS CloudFormation because CloudFormation's primary purpose is to provision and manage infrastructure resources in a predictable, repeatable way. Snapshots, however, are typically used for backup and disaster recovery purposes, which are operational tasks rather than provisioning tasks. While CloudFormation supports creating EBS volumes and attaching them to instances, automating snapshot creation is outside its core functionality.

Instead, snapshots are usually created:

1. Manually through the AWS Management Console or CLI.
2. Using automation tools like AWS Lambda, EventBridge, or Systems Manager.

How to Create Snapshots with AWS Lambda

Using AWS Lambda, you can automate the creation of EBS snapshots based on events (e.g., instance state changes) or a schedule (e.g., nightly backups). Below is a sample Lambda function to create snapshots for all EBS volumes attached to a running EC2 instance

Steps to Implement Snapshot Automation with Lambda

1. **Create a Lambda Function:**
 - Use the code above to create a Python-based Lambda function.
 - Ensure the function has proper logging for debugging.
2. **Attach IAM Role:**
 - Attach an IAM role to the Lambda function with the following permissions:
 - `ec2:DescribeInstances`
 - `ec2:DescribeVolumes`
 - `ec2:CreateSnapshot`
3. **Trigger the Function:**
 - **Option 1:** Use an **EventBridge Rule** to trigger the Lambda function on a schedule (e.g., nightly).
 - **Option 2:** Trigger the function based on **instance state changes** (e.g., when an instance stops or starts).
4. **Test the Function:**
 - Verify the function creates snapshots for all attached EBS volumes.
 - Use logging to ensure errors or skipped steps are identified.

Benefits of Using Lambda for Snapshot Automation

- **Flexibility:** Create snapshots based on events (like instance stop) or on a defined schedule.
- **Cost Optimization:** Automate snapshot cleanup to delete old snapshots after a retention period.
- **Scalability:** Easily handle snapshot automation for large environments without manual intervention.

Create function [Info](#)

Choose one of the following options to create your function.

☒ **Author from scratch**

Start with a simple Hello World example.

☐ **Use a blueprint**

Build a Lambda application from sample code and configuration presets for common use cases.

☐ **Container image**

Select a container image to deploy for your function.

Basic information

Function name

Enter a name that describes the purpose of your function.

snapshot-creation

Function name must be 1 to 64 characters, must be unique to the Region, and can't include spaces. Valid characters are a-z, A-Z, 0-9, hyphens (-), and underscores (_).

Runtime [Info](#)

Choose the language to use to write your function. Note that the console code editor supports only Node.js, Python, and Ruby.

Python 3.10



Architecture [Info](#)

Choose the instruction set architecture you want for your function code.

☒ x86_64

☐ arm64

Permissions [Info](#)

By default, Lambda will create an execution role with permissions to upload logs to Amazon CloudWatch Logs. You can customize this default role later when adding triggers.

snapshot-creation

Throttle

 Copy ARN

Actions ▼

▼ Function overview [Info](#)

Diagram

Template



snapshot-creation



Layers

(0)

+ Add trigger

+ Add destination

Export to Infrastructure Composer

Download ▼

Description

-

Last modified

1 hour ago

Function ARN

 arn:aws:lambda:us-east-1:600627341010:function:snapshot-creation

Function URL [Info](#)

-

[Code](#)

[Test](#)

[Monitor](#)

[Configuration](#)

[Aliases](#)

[Versions](#)

Created inline policy

[Review and create](#)

Policy details

Policy name

Enter a meaningful name to identify this policy.

Maximum 128 characters. Use alphanumeric and '+=, @-_' characters.

Permissions defined in this policy [Info](#)

[Edit](#)

Permissions defined in this policy document specify which actions are allowed or denied. To define permissions for an IAM identity (user, user group, or role), attach a policy to it

Allow (4 of 438 services)

☐ Show remaining 434 services

Service ▲	Access level ▼	Resource	Request condition
CloudWatch Logs	Limited: Write	region string like All	None
EC2	Limited: List, Write	All resources	None
EventBridge	Limited: Write	region string like All, RuleName string like All	None
Lambda	Limited: Write	FunctionName string like All, region string like All	None

Created cron expression in cloudwatch

Choose the schedule type that best meets your needs.



Cron-based schedule

A schedule set using a cron expression that runs at a specific time, such as 8:00 a.m. PST on the first Monday of every month.



Rate-based schedule

A schedule that runs at a regular rate, such as every 10 minutes.

Cron expression [Info](#)

Define the cron expression for the schedule



Copy



Clear

cron (

0/3

Minutes

*

Hours

*

Day of month

*

Month

?

Day of the week

*

Year

)

Next 10 trigger dates

Date and time are displayed in your current time zone in UTC format, e.g. "Wed, Nov 9, 2022 09:00 (UTC - 08:00)" for Pacific time

Sun, 22 Dec 2024 23:45:00 (UTC+05:30)

Sun, 22 Dec 2024 23:48:00 (UTC+05:30)

Sun, 22 Dec 2024 23:51:00 (UTC+05:30)

Sun, 22 Dec 2024 23:54:00 (UTC+05:30)

Sun, 22 Dec 2024 23:57:00 (UTC+05:30)

Mon, 23 Dec 2024 00:00:00 (UTC+05:30)

Mon, 23 Dec 2024 00:03:00 (UTC+05:30)

Mon, 23 Dec 2024 00:06:00 (UTC+05:30)

Mon, 23 Dec 2024 00:09:00 (UTC+05:30)

Mon, 23 Dec 2024 00:12:00 (UTC+05:30)

Flexible time window

If you choose a flexible time window, Scheduler invokes your schedule within the time window you specify. For example, if you choose 15 minutes, your schedule runs within 15 minutes after the schedule start time.

Select



- Step 1
Specify schedule detail
- Step 2 - optional
Select target
- Step 3 - optional
Settings
- Step 4
Review and create schedule

Review and create schedule

Step 1: Schedule detail

[Edit](#)

Schedule detail

Schedule name
lambda-crontab

Description
-

Schedule group
default

Time zone
(UTC+05:30) Asia/Calcutta

Occurrence
Recurring

Start date and time
-

End date and time
-

Flexible time window
5 minutes

Cron expression

0/3 * * * ? *
Minutes Hours Day of month Month Day of week Year

Next 10 trigger dates

Date and time are displayed in the selected time zone for which this schedule is set in UTC format, e.g. "Wed, Nov 9, 2022 09:00 (UTC - 08:00)"

Sun, 22 Dec 2024 23:45:00 (UTC+05:30)
 Sun, 22 Dec 2024 23:48:00 (UTC+05:30)
 Sun, 22 Dec 2024 23:51:00 (UTC+05:30)
 Sun, 22 Dec 2024 23:54:00 (UTC+05:30)
 Sun, 22 Dec 2024 23:57:00 (UTC+05:30)
 Mon, 23 Dec 2024 00:00:00 (UTC+05:30)
 Mon, 23 Dec 2024 00:03:00 (UTC+05:30)
 Mon, 23 Dec 2024 00:06:00 (UTC+05:30)
 Mon, 23 Dec 2024 00:09:00 (UTC+05:30)
 Mon, 23 Dec 2024 00:12:00 (UTC+05:30)

Step 2: Target

[Edit](#)

128 MB

Set memory to between 128 MB and 10240 MB

Ephemeral storage [Info](#)

You can configure up to 10 GB of ephemeral storage (/tmp) for your function. [View pricing](#)

512 MB

Set ephemeral storage (/tmp) to between 512 MB and 10240 MB.

SnapStart [Info](#)

Reduce startup time by having Lambda cache a snapshot of your function after the function has initialized. To evaluate whether your function code is resilient to snapshot operations, review the [SnapStart compatibility considerations](#). For Python and .NET runtimes, [view pricing](#).

None

Supported runtimes: .NET 8 (C#/F#/PowerShell), Java 11, Java 17, Java 21, Python 3.12, Python 3.13.

Timeout

1 min 0 sec

Execution role

Choose a role that defines the permissions of your function. To create a custom role, go to the [IAM console](#).

- ☒ Use an existing role
- ☐ Create a new role from AWS policy templates

Existing role

Choose an existing role that you've created to be used with this Lambda function. The role must have permission to upload logs to Amazon CloudWatch Logs.

service-role/snapshot-creation-role-zz3r2anc

[View the snapshot-creation-role-zz3r2anc role](#) on the IAM console.

Python code to create snapshots

EC2 S3 CodeDeploy CodePipeline RDS VPC IAM CloudFormation

Lambda > Functions > snapshot-creation

The test event New-cost-optimization was successfully saved.

lambda_function.py



DEPLOY

Deploy (Ctrl+Shift+U)

Test (Ctrl+Shift+I)

TEST EVENTS [SELECTED: NEW-COST-OPTIMIZATION]

+ Create new test event

Private saved events
New-cost-optimization



ENVIRONMENT VARIABLES

0 0 0 Amazon Q

```
1 import boto3
2 import logging
3
4 def lambda_handler(event, context):
5     # Initialize logging
6     logging.basicConfig(level=logging.INFO)
7     logger = logging.getLogger()
8
9     # EC2 instance ID and description
10    instance_id = "i-071f12f1416e9ad45"
11    description = f"Snapshot created for instance {instance_id}"
12
13    # Initialize the EC2 and CloudWatch services
14    ec2_client = boto3.client('ec2')
15
16    try:
17        # Log the event details
18        logger.info(f"Received event: {event}")
19
20        # Check for the state change in the event
21        for record in event.get('detail', {}).get('Records', []):
22            if record == instance_id and event['event'] == 'instance-state-change':
23                logger.info(f"Instance {instance_id} state changed to {event['detail']['State']}")
24
25                # Find the attached volumes
26                volumes = ec2_client.describe_volumes(Filters=[{'Name': 'attachment.instance_id', 'Values': [instance_id]}])
27
28                # Create snapshots for each volume
29                for volume in volumes['Volumes']:
30                    snapshot_id = ec2_client.create_snapshot(
31                        VolumeId=volume['VolumeId'],
32                        Description=description,
33                        Tags=[{'Key': 'Name', 'Value': f'Snapshot {volume["VolumeId"]'}])
34                    logger.info(f"Created snapshot {snapshot_id} for volume {volume['VolumeId']}")
35
36    except Exception as e:
37        logger.error(f"Error creating snapshots: {e}")
38    return {'statusCode': 200, 'body': 'Snapshots created successfully'}
```

Event Name

New-cost-optimization

Maximum of 25 characters consisting of letters, numbers, dots, hyphens and underscores.

Event sharing settings

Private

☒ This event is only available in the Lambda Console and to the event creator. You can configure a total of ten. [Learn more](#)

Shareable

☐ This event is available to IAM users within the same account who have permissions to access and use shareable events. [Learn more](#)

Template - optional

New-cost-optimization

Event JSON

```
1 {
2   "key1": "value1",
3   "key2": "value2",
4   "key3": "value3"
5 }
```

Test event is saved successfully.

Lambda Layout: US

output

≡ [Lambda](#) > [Functions](#) > snapshot-creation

Code source

Info

Upload from

← → snapshot-creation

EXPLORER

▼ SNAPSHOT-CREATION

lambda_function.py

▼ DEPLOY

Deploy (Ctrl+Shift+U)

Test (Ctrl+Shift+I)

▼ TEST EVENTS [SELECTED: NEW-COST-OPTIMIZATION]

lambda_function.py

```
4 def lambda_handler(event, context):
16     try:
24         ):
44         return {
47             }
48
49     # If the instance is not stopped, no action is taken
50     logger.info("No action taken")
51     return {
52         "statusCode": 200,
53         "body": "No action taken"
54     }
55
56 except Exception as e:
57     logger.error(f"Error while creating snapshot: {e}")
```

PROBLEMS

OUTPUT

CODE REFERENCE LOG

TERMINAL

Status: Succeeded

Test Event Name: New-cost-optimization

Response:

{

"statusCode": 200,

Execution Results

Now snapshot are created

EC2

S3

CodeDeploy

CodePipeline

RDS

VPC

IAM

CloudFormation

EC2 > Spot requests

Dashboard

EC2 Global View

Events

Instances

Instance Types

Launch Templates

Spot Requests

Savings Plans

Reserved Instances

Dedicated Hosts

Capacity Reservations

Images

AMIs

AMI Catalog

Elastic Block Store

Volumes

Snapshots

Lifecycle Manager

Successfully deleted snapshot snap-04b57008945d87a94.

Refresh

Recycle Bin

Actions

Create snapshot

Owned by me

Search

☐	Name	Snapshot ID	Volume size	Description	Storage tier	Snapshot status	Started
☐	snap-efs	snap-001297f45e1db5555	4 GiB	Snapshot created for insta...	Standard	Completed	2024/12/23 00:38 GM
☐	snap-root	snap-0d56e6b89efa257fe	8 GiB	Snapshot created for insta...	Standard	Completed	2024/12/23 00:38 GM

Select a snapshot above.

Creating trigger using event-bridge in lambda

☰ [Lambda](#) > Add triggers



EventBridge (CloudWatch Events)

aws asynchronous schedule management-tools

Rule

Pick an existing rule, or create a new one.

- ☒ Create a new rule
- ☐ Existing rules

Rule name

Enter a name to uniquely identify your rule.

event-trigger

Rule description

Provide an optional description for your rule.

Rule type

Trigger your target based on an event pattern, or based on an automated schedule.

- ☒ Event pattern
- ☐ Schedule expression

EC2

EC2 instance state-change notification



Detail



EC2 instance state-change notification



Detail

Detail



Instances



State

Instances

Instance ID

i-071f12f1416e9ad45

Delete

Add Instance ID

Delete Last Instance ID

State

× stopped

✔ Successfully created the function **detect-delete**. You can now change its code and configuration. To invoke your function with a test event, choose "Test".

detect-delete

Throttle

📄 Copy ARN

Actions ▼

▼ Function overview [Info](#)

Diagram

Template



detect-delete



Layers

(0)

+ Add trigger

+ Add destination

Export to Infrastructure Composer

Download ▼

Description

-

Last modified

1 second ago

Function ARN

📄 `arn:aws:lambda:us-east-1:600627341010:function:detect-delete`

Function URL [Info](#)

-

[Code](#)

[Test](#)

[Monitor](#)

[Configuration](#)

[Aliases](#)

[Versions](#)

Events

Instances

Launch Templates

Spot Requests

Savings Plans

Reserved Instances

Dedicated Hosts

Capacity Reservations

AMIs

AMI Catalog

Volumes

Snapshots

Lifecycle Manager

Last updated
less than a minute ago

Connect

Instance state

Actions ▼

Launch instances

Q Find Instance by attribute or tag (case-sensitive)

All states ▼

☒	Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone	Public IPv4
☒	webapp	i-071f12f1416e9ad45	Shutting-d...	t2.micro	Initializing	View alarms +	us-east-1a	ec2-54-209

i-071f12f1416e9ad45 (webapp)

Details

Status and alarms

Monitoring

Security

Networking

Storage

Tags

▼ Instance summary [Info](#)

Instance ID

 i-071f12f1416e9ad45

IPv6 address

Hostname type

IP name: ip-172-31-24-91.ec2.internal

Public IPv4 address

54.209.6.224 | [open address](#)

Instance state

⌚ Shutting-down

Private IP DNS name (IPv4 only)

ip-172-31-24-91.ec2.internal

Private IPv4 addresses

172.31.24.91

Public IPv4 DNS

ec2-54-209-6-224.compute-1.amazonaws.com |
open address

When i deleted ec2 instance drift is showing on CFT

aws

Search

[Alt+S]

N. Virginia

kechiya

EC2

S3

CodeDeploy

CodePipeline

RDS

VPC

IAM

CloudFormation

CloudFormation

Stacks

stale-stack

Drifts

CloudFormation

<

Stacks

Stack details

Drifts

StackSets

Exports

Infrastructure Composer

IaC generator

Hooks overview

Hooks

Registry

Public extensions

Activated extensions

Publisher

Drifts

Detect stack drift

Stack drift status

Drift detection enables you to detect whether a stack's actual configuration differs, or has drifted, from its template configuration. [Learn more](#)

Drift status

⚠️ DRIFTED

Last drift check time
2024-12-23 01:04:54 UTC+0530

Only resources which currently support drift detection are displayed here. To view all of your stack resources, see your stack details page. [Learn more](#)

Resource drift status (3)

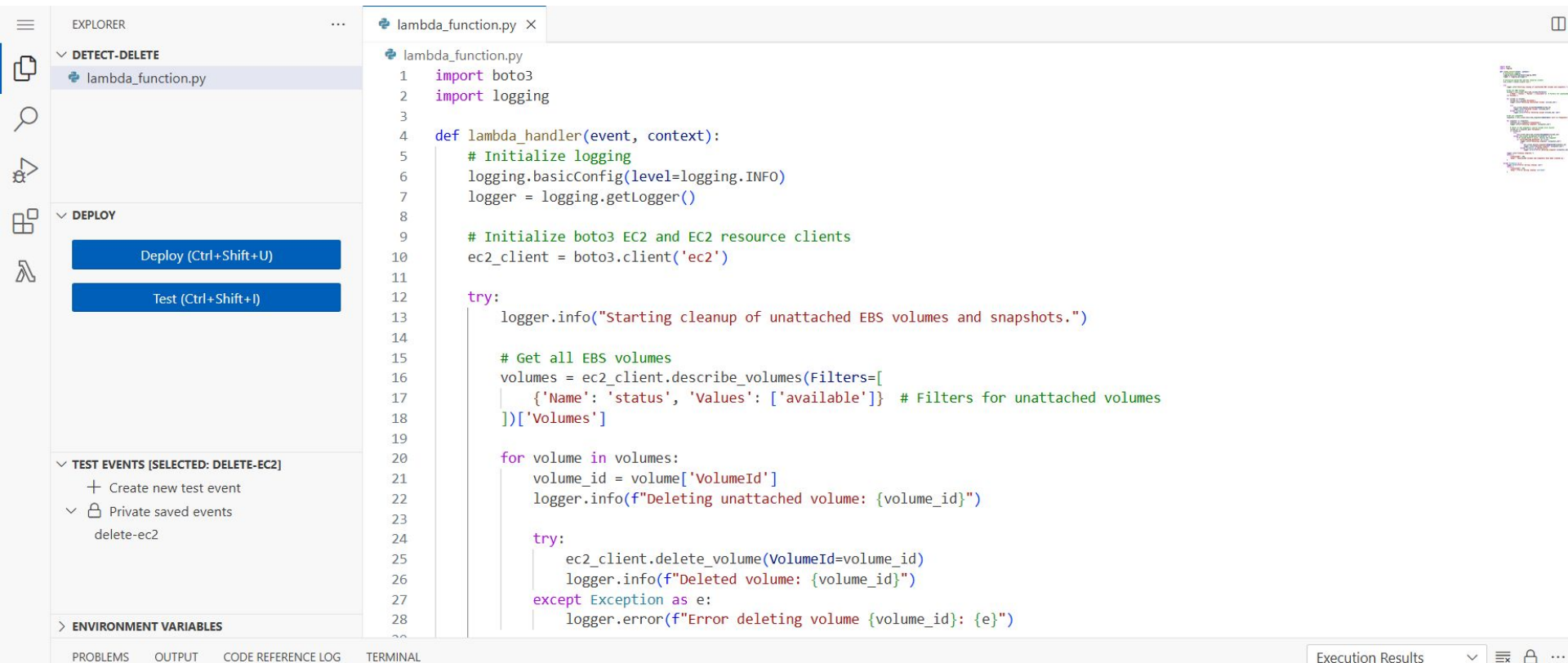
View drift details

Detect drift for resource

Search resources

	Logical ID	Physical ID	Type	Drift status	Timestamp	Module
☐	WebAppEBSVolume	vol-0ae60e8b916b8cf86	AWS::EC2::Volume	✅ IN_SYNC	2024-12-22 22:11:21 UTC+0530	-
☐	WebAppInstance	i-071f12f1416e9ad45	AWS::EC2::Instance	✅ IN_SYNC	2024-12-22 22:11:23 UTC+0530	-
☐	WebAppVolumeAttachm	vol-	AWS::EC2::VolumeAttach	-	2024-12-22 22:11:21	-

Python code to delete stale resources



The screenshot shows a code editor interface with a sidebar on the left and a main code area. The sidebar contains an 'EXPLORER' panel with a 'DETECT-DELETE' section containing a file named 'lambda_function.py'. Below this is a 'DEPLOY' section with two buttons: 'Deploy (Ctrl+Shift+U)' and 'Test (Ctrl+Shift+I)'. Further down is a 'TEST EVENTS [SELECTED: DELETE-EC2]' section with a '+ Create new test event' button and a 'Private saved events' section containing 'delete-ec2'. At the bottom of the sidebar is an 'ENVIRONMENT VARIABLES' section. The main code area displays the contents of 'lambda_function.py', which is a Python script for deleting unattached EBS volumes and snapshots. The code includes imports for 'boto3' and 'logging', a 'lambda_handler' function, and logic to find and delete unattached volumes. The bottom of the editor shows a 'PROBLEMS' panel, an 'OUTPUT' panel, a 'CODE REFERENCE LOG' panel, and a 'TERMINAL' panel. On the far right, there is an 'Execution Results' panel.

```
1 import boto3
2 import logging
3
4 def lambda_handler(event, context):
5     # Initialize logging
6     logging.basicConfig(level=logging.INFO)
7     logger = logging.getLogger()
8
9     # Initialize boto3 EC2 and EC2 resource clients
10    ec2_client = boto3.client('ec2')
11
12    try:
13        logger.info("Starting cleanup of unattached EBS volumes and snapshots.")
14
15        # Get all EBS volumes
16        volumes = ec2_client.describe_volumes(Filters=[
17            {'Name': 'status', 'Values': ['available']} # Filters for unattached volumes
18        ])['Volumes']
19
20        for volume in volumes:
21            volume_id = volume['VolumeId']
22            logger.info(f"Deleting unattached volume: {volume_id}")
23
24            try:
25                ec2_client.delete_volume(VolumeId=volume_id)
26                logger.info(f"Deleted volume: {volume_id}")
27            except Exception as e:
28                logger.error(f"Error deleting volume {volume_id}: {e}")
```

output

PROBLEMS OUTPUT CODE REFERENCE LOG TERMINAL

Execution Results

Status: Succeeded
Test Event Name: delete-ec2

Response:

```
{  
  "statusCode": 200,  
  "body": "Unattached volumes and snapshots have been cleaned up."  
}
```

Function Logs:

START RequestId: 525f2724-5228-446b-a468-2a47cf4e57d1 Version: \$LATEST

END RequestId: 525f2724-5228-446b-a468-2a47cf4e57d1

REPORT RequestId: 525f2724-5228-446b-a468-2a47cf4e57d1 Duration: 3591.24 ms Billed Duration: 3592 ms Memory Size: 128 MB Max Memory Used: 89 MB Init Duration: 279.16 ms

Request ID: 525f2724-5228-446b-a468-2a47cf4e57d1

Stale EBS volumes are deleted

EC2

S3

CodeDeploy

CodePipeline

RDS

VPC

IAM

CloudFormation

EC2

Spot requests

Dashboard

EC2 Global View

Events

Instances

Instance Types

Launch Templates

Spot Requests

Savings Plans

Reserved Instances

Dedicated Hosts

Capacity Reservations

Images

AMIs

AMI Catalog

Elastic Block Store

Volumes

Snapshots

Lifecycle Manager

Volumes

Info

Saved filter sets

Choose filter set

Search

Name

Volume ID

Type

Size

IOPS

Throughput

Snapshot ID

Created

You currently have no volumes in this region

Fault tolerance for all volumes in this Region

Snapshot summary

Recently backed up volumes / Total # volumes

0 / 2

Last updated on Mon, Dec 23, 2024, 12:12:25 AM (GMT+05:30)

Data Lifecycle Manager default policy for EBS Snapshots status

No default policy set up | Create policy

Stale snapshot are deleted

EC2

S3

CodeDeploy

CodePipeline

RDS

VPC

IAM

CloudFormation

☰

EC2 > Spot requests

Dashboard

EC2 Global View

Events

▼ Instances

Instances

Instance Types

Launch Templates

Spot Requests

Savings Plans

Reserved Instances

Dedicated Hosts

Capacity Reservations

▼ Images

AMIs

AMI Catalog

▼ Elastic Block Store

Volumes

Snapshots

Lifecycle Manager

Snapshots

Info

Owned by me ▼

Q Search

↺

Recycle Bin

Actions ▼

Create snapshot

◀

1

▶

⚙

☐	Name ▼	Snapshot ID ▼	Volume size ▼	Description ▼	Storage tier ▼	Snapshot status ▼	Started
You currently have no snapshots in this Region.							